Homework 02

Haoran Xu

2026-09-06

Question 1 (Training and test error under a fixed design)

Solution

Part a. *(average training and test MSE over 200 runs; monotonicity of training MSE.)* $\mathbf{X}_{\text{all}}$ is drawn once and held fixed; only ϵ and ϵ^* are redrawn in each of the 200 runs.

```
set.seed(43202)
n <- 100; p <- 20; nsim <- 200
X_all <- matrix(rnorm(n * p), n, p)
beta <- 0.4^sqrt(1:p)
mu <- as.vector(X_all %*% beta)

train_mat <- test_mat <- matrix(NA_real_, nsim, p + 1)   # columns are m = 0..p
for (b in 1:nsim) {
  y <- mu + rnorm(n)            # training response
  y_star <- mu + rnorm(n)       # independent test response, same mu
  for (m in 0:p) {
    Xm <- cbind(1, X_all[, seq_len(m), drop = FALSE])
    fit <- as.vector(Xm %*% qr.solve(Xm, y))
    train_mat[b, m + 1] <- mean((y - fit)^2)
    test_mat[b, m + 1] <- mean((y_star - fit)^2)
  }
}
avg_train <- colMeans(train_mat); avg_test <- colMeans(test_mat)

# training MSE is non-increasing in m within every one of the 200 runs
worst_step <- max(apply(train_mat, 1, function(r) max(diff(r))))
c(all_runs_non_increasing = all(apply(train_mat, 1, function(r)
    all(diff(r) <= 1e-8))), worst_increase = worst_step)
#> all_runs_non_increasing          worst_increase
#>             1.00000e+00              -4.12871e-11
```

Across all $200 \times 20 = 4000$ increments the largest change is -4.13×10^{-11}, i.e. never positive beyond rounding: adding a column enlarges $\mathcal{C}(\mathbf{X}_m)$, and the least-squares fit already available in the smaller space remains feasible, so RSS cannot rise. The average training curve falls monotonically from 1.315 to 0.795, while average test MSE falls to a minimum at $m = 6$ (1.094) and then climbs back to 1.217 — the classic U shape (plot in part c).

Part b. *(theoretical expectations against the simulation averages.)*

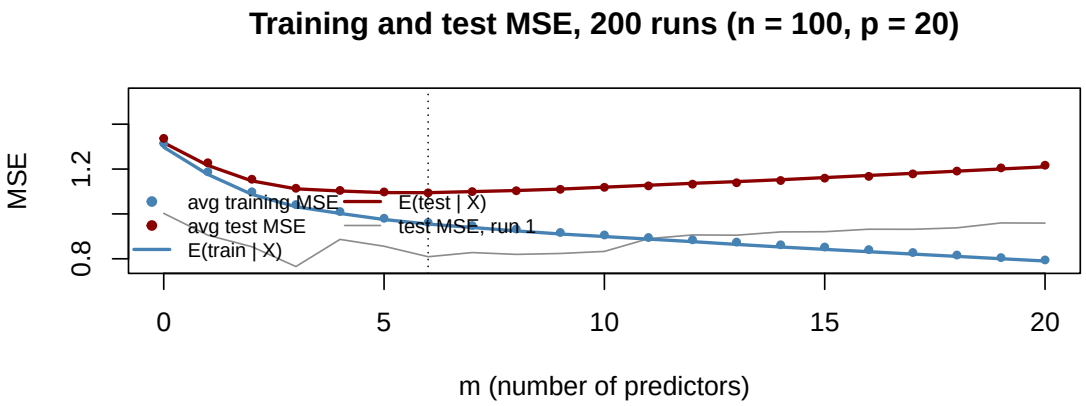
```
B2 <- sapply(0:p, function(m) {
  Xm <- cbind(1, X_all[, seq_len(m), drop = FALSE])
  sum((mu - Xm %*% qr.solve(Xm, mu))^2)          # ||(I - H_m) mu||^2
})
th_train <- B2 / n + 1 - (0:p + 1) / n
th_test  <- B2 / n + 1 + (0:p + 1) / n
```

```r
cmp <- data.frame(m = 0:p, B2_over_n = B2 / n,
                  avg_train, th_train, avg_test, th_test)
round(cmp[c(1, 2, 3, 5, 7, 8, 11, 16, 21), ], 4)
#>     m B2_over_n avg_train th_train avg_test th_test
#> 1   0    0.3074    1.3147   1.2974   1.3367  1.3174
#> 2   1    0.1963    1.1881   1.1763   1.2279  1.2163
#> 3   2    0.1172    1.0986   1.0872   1.1546  1.1472
#> 5   4    0.0522    1.0105   1.0022   1.1046  1.1022
#> 7   6    0.0247    0.9623   0.9547   1.0940  1.0947
#> 8   7    0.0195    0.9481   0.9395   1.0995  1.0995
#> 11 10    0.0092    0.9061   0.8992   1.1186  1.1192
#> 16 15    0.0019    0.8524   0.8419   1.1594  1.1619
#> 21 20    0.0000    0.7948   0.7900   1.2170  1.2100
c(max_abs_gap_train = max(abs(avg_train - th_train)),
  max_abs_gap_test = max(abs(avg_test - th_test)),
  mc_se_train = max(apply(train_mat, 2, sd)) / sqrt(nsim),
  mc_se_test = max(apply(test_mat, 2, sd)) / sqrt(nsim))
#> max_abs_gap_train  max_abs_gap_test       mc_se_train        mc_se_test
#>         0.0172388         0.0192910         0.0124093         0.0126525
```

The two theoretical curves track the simulation averages closely: the largest gap is 0.0172 (training) and 0.0193 (test), each roughly one Monte Carlo standard error of a 200-run average, so the residual discrepancy is simulation noise rather than a systematic error. The mean squared approximation bias B_m^2/n falls from 0.307 at $m = 0$ to 0 at $m = 20$, and does the same work in both formulas; the two expectations differ only through the sign of the estimation term $\pm(m+1)/n$. Since $\beta_j = 0.4^{\sqrt{j}}$ decays quickly, most of the bias is removed by the first few columns, after which $\pm(m+1)/n$ dominates.

Part c. *(a single-run test curve; why it is rougher, and why training MSE alone cannot pick m.)*

```r
m_grid <- 0:p
plot(m_grid, test_mat[1, ], type = "l", col = "grey55", lwd = 1,
     ylim = range(train_mat[1, ], avg_test, th_test, test_mat[1, ]),
     xlab = "m (number of predictors)", ylab = "MSE",
     main = "Training and test MSE, 200 runs (n = 100, p = 20)")
lines(m_grid, th_train, col = "steelblue", lwd = 2)
lines(m_grid, th_test, col = "darkred", lwd = 2)
points(m_grid, avg_train, pch = 19, cex = 0.6, col = "steelblue")
points(m_grid, avg_test, pch = 19, cex = 0.6, col = "darkred")
abline(v = which.min(avg_test) - 1, lty = 3)
legend("bottomleft", bty = "n", cex = 0.75, ncol = 2,
       legend = c("avg training MSE", "avg test MSE",
                  "E(train | X)", "E(test | X)", "test MSE, run 1"),
       pch = c(19, 19, NA, NA, NA), lty = c(NA, NA, 1, 1, 1),
       lwd = c(NA, NA, 2, 2, 1),
       col = c("steelblue", "darkred", "steelblue", "darkred", "grey55"))
```



Why one run is rough. Expanding a single realization,

$$\mathrm{MSE}_{\mathrm{test},m} = \underbrace{\frac{B_m^2}{n}}_{\text{smooth}} + \underbrace{\frac{\|\epsilon^*\|^2}{n}}_{\text{same for all } m} + \frac{\epsilon^{\mathsf{T}}\mathbf{H}_m\epsilon}{n} + \frac{2}{n}\mu^{\mathsf{T}}(\mathbf{I}-\mathbf{H}_m)\epsilon^* - \frac{2}{n}\epsilon^{*\mathsf{T}}\mathbf{H}_m\epsilon.$$

Only the first term is what the plot is meant to display. The last three are mean-zero fluctuations that jump each time a new column enters $\mathbf{H}_m$; with $\sigma = 1$ their standard deviations are of order $2B_m/n \approx 0.11$ and $2\sqrt{m+1}/n \approx 0.09$, comparable to the whole 0.22 vertical range of the expected curve, so a single run zig-zags and its minimizer ($m = 3$ here) is essentially arbitrary. Averaging 200 independent runs divides those fluctuations by $\sqrt{200} \approx 14$, leaving the smooth U.

Why training MSE cannot select m. The two expectations share B_m^2/n but carry the estimation term with opposite signs:

$$E(\mathrm{MSE}_{\mathrm{train},m}) = \frac{B_m^2}{n} + 1 - \frac{m+1}{n}, \qquad E(\mathrm{MSE}_{\mathrm{test},m}) = \frac{B_m^2}{n} + 1 + \frac{m+1}{n}.$$

Both terms of the training curve are non-increasing in m — bias falls *and* the fit soaks up an extra σ^2/n of noise per added coefficient — so training MSE decreases even when the added column is pure noise, and its minimizer is always the largest model. The test curve pays $+\sigma^2(m+1)/n$ instead of being credited $-\sigma^2(m+1)/n$, so it turns upward once bias reduction is exhausted; here it bottoms at $m = 6$. The gap between them, the expected optimism $2\sigma^2(m+1)/n$, is exactly the quantity training error cannot see, which is why an explicit penalty (C_p, AIC, BIC) or held-out data is needed.

Question 2 (Prediction error at one target point)

Solution

Part a. *(μ_0, squared bias and expected squared error; where the error moves.)*

```
n <- 100; p <- 6; sigma2 <- 1
X_all <- outer(1:n, 1:p, function(i, j) sqrt(2) * cos(pi * j * (i - 0.5) / n))
c(max_col_sum = max(abs(colSums(X_all))),
  max_gram_dev = max(abs(crossprod(X_all) / n - diag(p))))   # design checks
#>  max_col_sum max_gram_dev
#>  1.53211e-14  4.44089e-16

beta <- rep(0.5, p); x0 <- c(0, 1, 0, 0, 1, 0)
mu0 <- sum(x0 * beta)
bias2 <- sapply(0:p, function(m)
  if (m >= p) 0 else sum(x0[(m + 1):p] * beta[(m + 1):p])^2)
```

```
varpt <- sapply(0:p, function(m) sigma2 / n * (1 + sum(x0[seq_len(m)]^2)))
theory <- data.frame(m = 0:p, bias2, variance = varpt, expected = bias2 + varpt)
c(mu0 = mu0); theory
#> mu0
#>   1
#>   m bias2 variance expected
#> 1 0  1.00     0.01     1.01
#> 2 1  1.00     0.01     1.01
#> 3 2  0.25     0.02     0.27
#> 4 3  0.25     0.02     0.27
#> 5 4  0.25     0.02     0.27
#> 6 5  0.00     0.03     0.03
#> 7 6  0.00     0.03     0.03
```

Reading the formula. The model with m predictors estimates $\beta_1, \dots, \beta_m$ and silently sets $\beta_{m+1}, \dots, \beta_p$ to zero, so its target is $\sum_{j \le m} x_{0j}\beta_j$ rather than μ_0; the shortfall $\sum_{j>m} x_{0j}\beta_j$ is a *fixed* offset, and squaring it gives the bias term. The second term is the sampling variance of $\hat{\mu}_{0,m}$. Because $\frac{1}{n}\mathbf{X}_{\text{all}}^\top \mathbf{X}_{\text{all}} = \mathbf{I}_p$ and $\mathbf{1}^\top \mathbf{X}_{\text{all}} = \mathbf{0}^\top$, the intercept $\bar{y}$ and each $\hat{\beta}_j = \frac{1}{n}\mathbf{x}_j^\top \mathbf{y}$ are uncorrelated with common variance σ^2/n, so $\text{Var}(\hat{\mu}_{0,m}) = \frac{\sigma^2}{n}(1 + \sum_{j \le m} x_{0j}^2)$: one σ^2/n for the intercept and $x_{0j}^2\sigma^2/n$ for each estimated slope. Bias falls (weakly) and variance rises (weakly) with m — the bias–variance trade-off at a single point.

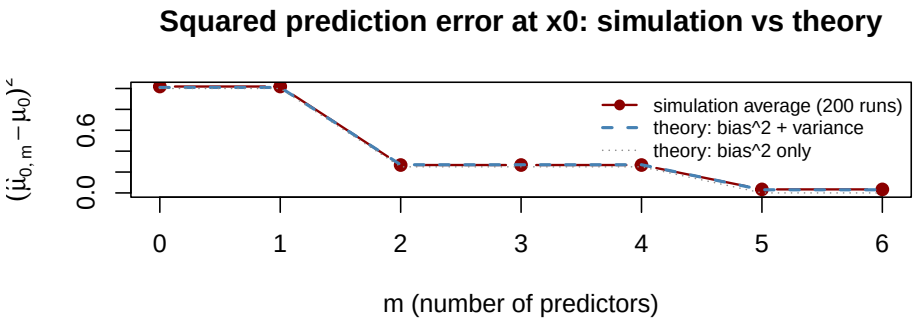
Here $\mu_0 = 1$ and the expected error changes only at $m = 2$ ($1.01 \to 0.27$, when x_2 removes half the bias) and $m = 5$ ($0.27 \to 0.03$, when x_5 removes the rest). It is unchanged at $m = 1, 3, 4, 6$, because those columns have $x_{0j} = 0$: they neither shrink the bias sum nor enlarge $\sum_{j \le m} x_{0j}^2$. The minimum 0.03 is first attained at $m = 5$.

Part b. *(200 simulations against the theoretical curve.)*

```
set.seed(43203)
nsim <- 200
mu <- as.vector(X_all %*% beta)
err <- matrix(NA_real_, nsim, p + 1)
for (b in 1:nsim) {
  y <- mu + rnorm(n)
  for (m in 0:p) {
    Xm <- cbind(1, X_all[, seq_len(m), drop = FALSE])
    pred <- sum(c(1, x0[seq_len(m)]) * qr.solve(Xm, y))
    err[b, m + 1] <- (pred - mu0)^2
  }
}
sim_avg <- colMeans(err)
round(data.frame(m = 0:p, simulation = sim_avg, theory = bias2 + varpt,
                 mc_se = apply(err, 2, sd) / sqrt(nsim)), 4)
#>   m simulation theory  mc_se
#> 1 0     1.0189   1.01 0.0143
#> 2 1     1.0189   1.01 0.0143
#> 3 2     0.2671   0.27 0.0105
#> 4 3     0.2671   0.27 0.0105
#> 5 4     0.2671   0.27 0.0105
#> 6 5     0.0338   0.03 0.0034
#> 7 6     0.0338   0.03 0.0034
c(smallest_minimizer = (0:p)[which.min(bias2 + varpt)])
#> smallest_minimizer
#>                  5
```

```
plot(0:p, sim_avg, type = "b", pch = 19, col = "darkred", lwd = 1.5,
     ylim = c(0, max(sim_avg, bias2 + varpt)),
     xlab = "m (number of predictors)",
     ylab = expression((hat(mu)[list(0, m)] - mu[0])^2),
```

```
     main = "Squared prediction error at x0: simulation vs theory")
lines(0:p, bias2 + varpt, col = "steelblue", lwd = 2, lty = 2)
lines(0:p, bias2, col = "grey55", lwd = 1, lty = 3)
legend("topright", bty = "n", cex = 0.75,
       legend = c("simulation average (200 runs)", "theory: bias^2 + variance",
                  "theory: bias^2 only"),
       lty = c(1, 2, 3), pch = c(19, NA, NA), lwd = c(1.5, 2, 1),
       col = c("darkred", "steelblue", "grey55"))
```



The simulation averages sit on the theoretical curve within one Monte Carlo standard error at every m (largest gap 0.0089), reproducing the same three-level staircase. The smallest m minimizing the theoretical error is $m = 5$; note that $m = 6$ ties with it in this design, so the extra predictor buys nothing. The simulated values for $m \in \{0, 1\}$, $\{2, 3, 4\}$ and $\{5, 6\}$ are *identical*, not merely close: with an orthogonal design $\hat{\beta}_j = \frac{1}{n}\mathbf{x}_j^\top \mathbf{y}$ does not depend on m, so adding a column with $x_{0j} = 0$ leaves $\hat{\mu}_{0,m}$ numerically unchanged.

Part c. *(why only predictors 2 and 5 matter here, and the contrast with Question 1.)* The mean response at the target is $\mu_0 = \mathbf{x}_0^\top \beta = \sum_{j=1}^{6} x_{0j}\beta_j$, and $x_{0j} = 0$ except for $j = 2, 5$. A nonzero β_j matters *at a point* only through the product $x_{0j}\beta_j$: predictors 1, 3, 4, 6 are switched off at $\mathbf{x}_0$, so however large their coefficients are they move neither μ_0 nor $\hat{\mu}_{0,m}$. They are irrelevant here but not globally — they shape the mean at other rows.

The contrast with Question 1 is exactly this locality. Averaging the same decomposition over all rows of the design gives $\frac{1}{n}\sum_i \frac{\sigma^2}{n}(1 + \sum_{j \le m} x_{ij}^2) = \frac{\sigma^2(m+1)}{n}$, using $\frac{1}{n}\sum_i x_{ij}^2 = 1$, and the averaged squared bias is B_m^2/n — the Question 1 formula. So test MSE weights every predictor by its *average* contribution and every nonzero β_j helps, producing a smooth U with an interior optimum. The target-specific error weights predictor j by x_{0j}^2 at this one point, so it is a step function: flat where $x_{0j} = 0$, dropping only at $j = 2$ and $j = 5$. A model can be excellent on average yet biased at a particular $\mathbf{x}_0$, and vice versa; the best model depends on where the prediction is wanted.

Question 3 (The optimism correction)

Solution

Part a. *(deriving the two conditional expectations.)* Write $\mathbf{y} = \mu + \epsilon$ and let $\mathbf{M} = \mathbf{I}_n - \mathbf{H}$, which is symmetric and idempotent with $\operatorname{tr}(\mathbf{M}) = n - (p + 1)$. Everything below is conditional on $\mathbf{X}$.

Training. $\mathbf{y} - \mathbf{H}\mathbf{y} = \mathbf{M}\mathbf{y} = \mathbf{M}\mu + \mathbf{M}\epsilon$, so

$$n\,\mathrm{MSE}_{\mathrm{train}} = \|\mathbf{M}\mu\|^2 + 2\mu^\top \mathbf{M}\epsilon + \epsilon^\top \mathbf{M}\epsilon,$$

using $\mathbf{M}^\top\mathbf{M} = \mathbf{M}$. Taking expectations, the first term is the constant B^2, the cross term vanishes because $E(\epsilon) = \mathbf{0}$, and $E(\epsilon^\top \mathbf{M}\epsilon) = \sigma^2 \operatorname{tr}(\mathbf{M}) = \sigma^2\{n - (p + 1)\}$. Dividing by n,

$$E(\mathrm{MSE}_{\mathrm{train}} \mid \mathbf{X}) = \frac{B^2}{n} + \sigma^2\left(1 - \frac{p+1}{n}\right).$$

Test. Since $\mathbf{y}^* = \mu + \epsilon^*$ and $\mathbf{H}\mathbf{y} = \mathbf{H}\mu + \mathbf{H}\epsilon$,

$$\mathbf{y}^* - \mathbf{H}\mathbf{y} = \mathbf{M}\mu + \epsilon^* - \mathbf{H}\epsilon.$$

Expanding the squared norm gives three squares and three cross products. Two cross products vanish in expectation because ϵ^* and ϵ are independent with mean zero ($E(\epsilon^{*\mathsf{T}}\mathbf{H}\epsilon) = E(\epsilon^*)^\mathsf{T}\mathbf{H}\,E(\epsilon) = 0$), and the third vanishes identically, since $(\mathbf{M}\mu)^\mathsf{T}\mathbf{H} = \mu^\mathsf{T}\mathbf{M}\mathbf{H} = \mathbf{0}^\mathsf{T}$ by $\mathbf{H}^2 = \mathbf{H}$. With $E\|\epsilon^*\|^2 = n\sigma^2$ and $E(\epsilon^\mathsf{T}\mathbf{H}\epsilon) = \sigma^2\,\mathrm{tr}(\mathbf{H}) = \sigma^2(p+1)$,

$$E(n\,\mathrm{MSE}_{\text{test}} \mid \mathbf{X}) = B^2 + n\sigma^2 + (p+1)\sigma^2 \implies E(\mathrm{MSE}_{\text{test}} \mid \mathbf{X}) = \frac{B^2}{n} + \sigma^2\left(1 + \frac{p+1}{n}\right).$$

The mechanism is visible in the two quadratic forms: fitting projects ϵ onto a $(p+1)$-dimensional subspace and *subtracts* that part from the training residual, while the test residual *adds* it back, since $\mathbf{H}\epsilon$ is noise the test response does not share.

Part b. *(expected optimism, and a numerical case.)* Subtracting,

$$E(\mathrm{MSE}_{\text{test}} - \mathrm{MSE}_{\text{train}} \mid \mathbf{X}) = \frac{2(p+1)\sigma^2}{n},$$

free of B^2: the optimism depends on the number of fitted coefficients and the noise level, not on how well the model approximates μ.

```
n3 <- 120; p3 <- 4; B2n <- 0.04; s2 <- 1
e_train <- B2n + s2 * (1 - (p3 + 1) / n3)
e_test  <- B2n + s2 * (1 + (p3 + 1) / n3)
c(E_train = e_train, E_test = e_test, optimism = e_test - e_train,
  formula_check = 2 * (p3 + 1) * s2 / n3)
#>       E_train        E_test      optimism formula_check
#>     0.9983333     1.0816667     0.0833333     0.0833333
```

So $E(\mathrm{MSE}_{\text{train}}) = 0.9983$, $E(\mathrm{MSE}_{\text{test}}) = 1.0817$, and the optimism is $2 \cdot 5/120 = 0.0833$. These are *expectations*. For one realized pair $(\mathbf{y}, \mathbf{y}^*)$ the difference is random with mean 0.0833 but a much larger spread: it contains $(\|\epsilon^*\|^2 - \|\epsilon\|^2)/n$ alone, of standard deviation $\sigma^2\sqrt{4/n} \approx 0.18$ under normal errors. A large training-noise draw or a mild test draw easily makes the realized test MSE the smaller of the two; the ordering is a long-run statement, not a guarantee for any single data set.

Part c. *(the corrected test MSE and C_p.)*

```
RSS <- 116.4; s2hat <- 0.96
mse_hat <- (RSS + 2 * (p3 + 1) * s2hat) / n3
Cp <- RSS / s2hat - n3 + 2 * (p3 + 1)
c(MSE_test_hat = mse_hat, Cp = Cp,
  identity_check = (s2hat / n3) * (Cp + n3))
#>   MSE_test_hat             Cp identity_check
#>           1.05          11.25           1.05
```

$\widehat{\mathrm{MSE}}_{\text{test}} = (116.4 + 9.6)/120 = 1.05$ and $C_p = 121.25 - 120 + 10 = 11.25$. They are the same quantity on different scales: dividing the C_p definition by n and multiplying by $\hat\sigma^2$,

$$\frac{\hat\sigma^2}{n}C_p = \frac{\mathrm{RSS} + 2(p+1)\hat\sigma^2}{n} - \hat\sigma^2 = \widehat{\mathrm{MSE}}_{\text{test}} - \hat\sigma^2, \qquad \text{i.e.} \qquad \widehat{\mathrm{MSE}}_{\text{test}} = \hat\sigma^2 + \frac{\hat\sigma^2}{n}\,C_p,$$

confirmed numerically above ($0.008 \times 131.25 = 1.05$). Both are the sample version of part (a): RSS/n estimates $E(\mathrm{MSE}_{\text{train}})$ and $2(p+1)\hat\sigma^2/n$ adds back the optimism. When n and $\hat\sigma^2$ are held common across candidate models — as they are when the variance is estimated once from a full reference model — the map $C_p \mapsto \hat\sigma^2 + (\hat\sigma^2/n)C_p$ is a strictly increasing affine transformation with slope $\hat\sigma^2/n > 0$. A strictly increasing transformation preserves ordering, so the two criteria rank every pair of models identically and share the same minimizer. This fails if $\hat\sigma^2$ is re-estimated within each candidate model, since the slope and intercept would then vary across models.

Question 4 (Comparing Mallows' C_p, AIC, and BIC)

Solution

Part a. *(the three fits, and the common variance estimate.)*

```
diab <- read.csv("data/diabetes.csv")
train <- diab[1:370, ]; n4 <- nrow(train)
pred_A <- c("bmi", "bp", "s5", "sex", "s1", "s2", "s4")
pred_B <- c(pred_A, "s6")
pred_full <- c("age", "sex", "bmi", "bp", "s1", "s2", "s3", "s4", "s5", "s6")

fit_it <- function(vars) lm(reformulate(vars, "y"), data = train)
fits <- lapply(list(A = pred_A, B = pred_B, full = pred_full), fit_it)
RSS <- sapply(fits, function(f) sum(resid(f)^2))
df_full <- n4 - (10 + 1)
sigma2_hat <- RSS[["full"]] / df_full

data.frame(model = c("A", "B", "full"), p = c(7, 8, 10), RSS = RSS,
           row.names = NULL)
#>   model  p     RSS
#> 1     A  7 1098909
#> 2     B  8 1089717
#> 3  full 10 1089452
c(n = n4, df_full = df_full, sigma2_hat = sigma2_hat,
  check_sigma_from_lm = summary(fits$full)$sigma^2)
#>                   n             df_full          sigma2_hat
#>              370.00              359.00             3034.68
#> check_sigma_from_lm
#>             3034.68
```

The full reference model has 359 residual degrees of freedom and gives $\hat{\sigma}^2 = 3034.68$, which matches the residual variance from `lm`. This single estimate is used for both candidates, as required.

Part b. *(the three criteria.)*

```
crit <- function(rss, p) c(
  Cp = rss / sigma2_hat - n4 + 2 * (p + 1),
  AIC_red = n4 * log(rss / n4) + 2 * (p + 1),
  BIC_red = n4 * log(rss / n4) + (p + 1) * log(n4))
tab <- rbind(A = crit(RSS[["A"]], 7), B = crit(RSS[["B"]], 8))
round(tab, 3)
#>      Cp AIC_red BIC_red
#> A 8.116 2974.64 3005.95
#> B 7.087 2973.53 3008.75
apply(tab, 2, function(col) rownames(tab)[which.min(col)])   # selection
#>      Cp AIC_red BIC_red
#>     "B"     "B"     "A"
```

C_p prefers **Model B** (7.087 vs 8.116), reduced AIC also prefers **Model B** (2973.53 vs 2974.64), and reduced BIC prefers **Model A** (3005.95 vs 3008.75). The three columns are on different scales and are compared only within a column.

Part c. *(gain versus penalty, and what selection does not establish.)*

```
gain_Cp  <- (RSS[["A"]] - RSS[["B"]]) / sigma2_hat   # Cp scale
gain_ic  <- n4 * log(RSS[["A"]] / RSS[["B"]])        # AIC/BIC scale
c(delta_RSS = RSS[["A"]] - RSS[["B"]], gain_Cp = gain_Cp,
  penalty_Cp = 2, gain_AIC_BIC = gain_ic,
  penalty_AIC = 2, penalty_BIC = log(n4))
#>    delta_RSS      gain_Cp   penalty_Cp gain_AIC_BIC  penalty_AIC
```

```
#>   9191.50885      3.02882      2.00000      3.10778      2.00000
#>  penalty_BIC
#>      5.91350
```

Adding `s6` reduces RSS by 9191.5, worth $\Delta\,\mathrm{RSS}\,/\hat{\sigma}^2 = 3.029$ on the C_p scale and $n\log(\mathrm{RSS}_A\,/\,\mathrm{RSS}_B) = 3.108$ on the AIC/BIC scale (the two agree closely because $n\log(1+x) \approx nx$ for small x). Each criterion charges one extra parameter:

- C_p charges 2, and $3.03 > 2$, so B wins by 1.029;
- AIC charges 2, and $3.11 > 2$, so B wins by 1.108;
- BIC charges $\log(370) = 5.914$, and $3.11 < 5.91$, so A wins by 2.806.

The disagreement is therefore entirely about the price of a parameter, not about the fit: the improvement is real but modest — just above the C_p/AIC threshold of 2 and well below BIC's $\log n$. C_p and AIC estimate out-of-sample prediction error and accept any variable expected to pay for itself in prediction; BIC's penalty grows with n, which makes it consistent for the true model under its assumptions but deliberately conservative. Near the boundary they naturally split.

Selecting a model does not establish that it generates the data. Each criterion only ranks the *candidates supplied to it*, under fixed assumptions (linearity, homoscedastic independent errors, and here a $\hat{\sigma}^2$ borrowed from a model that may itself be wrong); the true mean function need not be in the list, and the diabetes predictors are strongly correlated, so several models fit almost identically and the winner shifts with small perturbations of the data. What we can say is that B is the better *predictor* by two of the three criteria and A is the more parsimonious description favoured by the third — not that either is true.

Question 5 (Validation and final test data)

Solution

```
test5 <- diab[371:442, ]
ord <- c("age", "sex", "bmi", "bp", "s1", "s2", "s3", "s4", "s5", "s6")
fit_m <- function(m, d) lm(reformulate(if (m) ord[1:m] else "1", "y"), data = d)
mse <- function(f, d) mean((d$y - predict(f, d))^2)

test_mse <- sapply(0:10, function(m) mse(fit_m(m, train), test5))
round(setNames(test_mse, paste0("m=", 0:10)), 1)
#>    m=0    m=1    m=2    m=3    m=4    m=5    m=6    m=7    m=8    m=9
#> 5822.9 5527.8 5527.2 3719.2 3246.8 3234.9 3207.6 2636.5 2624.9 2369.3
#>   m=10
#> 2484.8
c(analyst_choice = (0:10)[which.min(test_mse)], analyst_value = min(test_mse))
#> analyst_choice  analyst_value
#>           9.00        2369.34
```

Part a. *(where the misuse starts, and why the minimum is too optimistic.)* The misuse begins at **step 2**. Step 1 is legitimate — the coefficients are fitted from training rows only. But step 2 evaluates *all eleven* candidates on the final test data, and in this procedure that comparison is what determines the model: the test set is being used as a validation set. Step 3 merely records the comparison and step 4 reports it, so the damage is already done at step 2. Computing a test MSE is harmless only for a model fixed before the test responses were seen; here the eleven values exist precisely in order to be ranked. Once the final test data have influenced which model is reported, they are no longer held out for it.

Each $\mathrm{MSE}_{\mathrm{test},m}$ is a noisy estimate of the true error of model m (here on only 72 rows), and $E[\min_m \widehat{E}_m] \le \min_m E[\widehat{E}_m]$ by Jensen's inequality applied to the concave minimum function: the minimum of several noisy estimates is biased *downward*, and the model that wins tends to be the one whose noise happened to be most favourable — the winner's curse. With eleven correlated candidates, part of the reported 2369.3 is genuine predictive quality and part is a lucky draw of the 72 test rows.

```r
set.seed(43205)                          # how large is that selection bias here?
R <- 300; gap <- numeric(R)
for (r in 1:R) {
  idx <- sample(nrow(diab))
  trn <- diab[idx[1:298], ]; sel <- diab[idx[299:370], ]; fresh <- diab[idx[371:442], ]
  f <- lapply(0:10, fit_m, d = trn)
  m_sel <- sapply(f, mse, d = sel)
  k <- which.min(m_sel)
  gap[r] <- mse(f[[k]], fresh) - m_sel[k]      # fresh error minus reported minimum
}
c(mean_gap = mean(gap), se = sd(gap) / sqrt(R))
#> mean_gap       se
#> 129.7464  37.4982
```

Repeating the analyst's own procedure 300 times on random splits of the same data — select m on one 72-row block, then score that model on a *fresh* 72-row block — the fresh error exceeds the reported minimum by 129.7 on average (Monte Carlo SE 37.5), about 5.5% of the reported value. The optimism is real, if modest at this size.

Part b. *(the same analysis done with five-fold cross-validation.)* Split the **370 training rows only** into five folds. For each $m = 0, \dots, 10$ and each fold k, fit on the other four folds and record the MSE on fold k; average the five to get CV_m. Choose $m^\star = \arg\min_m \mathrm{CV}_m$ — the predictor count is selected entirely inside the training data. Then **refit that single model on all 370 training rows** (all coefficients re-estimated on the full training set, no fold-specific fit is kept). Only then, once, are rows 371–442 touched, to report the prediction error of that one refitted model. The test set is never involved in choosing m.

```r
set.seed(43206)
folds <- sample(rep(1:5, length.out = nrow(train)))
cv <- sapply(0:10, function(m) mean(sapply(1:5, function(k) {
  mse(fit_m(m, train[folds != k, ]), train[folds == k, ])
})))
round(setNames(cv, paste0("m=", 0:10)), 1)
#>    m=0    m=1    m=2    m=3    m=4    m=5    m=6    m=7    m=8    m=9
#> 6015.3 5861.9 5904.2 4068.7 3824.9 3863.3 3885.2 3321.8 3353.9 3227.7
#>   m=10
#> 3212.8

m_star <- (0:10)[which.min(cv)]
final_fit <- fit_m(m_star, train)                  # refit on all 370 rows
c(m_star = m_star, cv_min = min(cv),
  honest_test_mse = mse(final_fit, test5))
#>          m_star          cv_min honest_test_mse
#>           10.00         3212.77         2484.78
```

Cross-validation selects $m^\star = 10$, and the honest estimate of its prediction error, computed once on the untouched final test set, is 2484.8 — larger than the analyst's 2369.3, as expected. (The CV curve is flat over $m \in \{9, 10\}$, so the fold split can tip the choice between the two; repeating the CV over several random fold assignments, or a one-standard-error rule, is the usual safeguard.)

Part c. *(what remains reportable, and what new evaluation would require.)* Having looked at all eleven test MSEs, the analyst cannot un-see them, but can still report honestly:

- the **whole table** of eleven test MSEs, labelled an exploratory comparison rather than the error of a selected model, disclosing that all eleven were inspected and that the reported minimum is the smallest of eleven noisy numbers;
- the **cross-validation curve** and the model it selects — CV never used the test rows, so it remains a valid selection rule and $\mathrm{CV}_{m^\star}$ a valid (if slightly optimistic, being itself a minimum) estimate;
- the **test MSE of any model fixed in advance** of seeing the table, e.g. the full $m = 10$ model, 2484.8;
- training MSEs, coefficients, and the C_p/AIC/BIC comparison of Question 4, none of which touch the test rows.

What cannot be reported as an unbiased estimate is the minimum 2369.3 attached to the selected model $m = 9$. A new final evaluation needs **data that have played no part in fitting or selection**: a fresh sample from the same population, or a portion of the existing data sequestered before any of this analysis and never examined. Re-splitting the same 442 rows does not fix it, since every row has now informed the analyst's choices; the clean remedy is to pre-register the procedure (CV inside training, one final evaluation) and apply it to genuinely new data.